

I'll shape this as a production-ready product plan, not just an idea: core features, architecture, call flow, AI agents, database, tech stack, security, deployment, and MVP-to-production roadmap.

Thought for 26s >

Below is a **production-level blueprint** for your idea: an **AI Customer Call Handling Software** that can receive customer calls, talk naturally, understand issues, solve them using company data/tools, create tickets, and transfer to a human when needed.

1. Project Idea Name

AI Voice Customer Support Agent

This software will work like a smart call center agent.

Customer calls your company number → AI answers → understands the problem → checks customer/ticket/order/policy data → gives answer or performs action → creates ticket/report → escalates to human when needed.

Example use cases:

- Insurance claim status call
 - Bank loan/application status call
 - E-commerce order tracking
 - SaaS customer support
 - Appointment booking
 - Complaint registration
 - Payment reminder support
 - Government citizen helpline POC
-

2. What the System Should Do

Your AI call agent should not only “talk”. It should behave like a real support executive.

Core Features

1. Receive phone calls

Customer calls a normal phone number. The system connects the live call audio to your AI backend.

You can use telephony providers like Twilio. Twilio Media Streams supports real-time call audio over WebSockets, and bidirectional Media Streams allow your backend to send audio back into the live call. Twilio explicitly mentions this can be used for real-time voice conversation with an AI assistant.  Twilio +1

2. Real-time voice conversation

The AI should listen and respond naturally, with low delay.

There are two common approaches:

Approach A: Speech-to-speech AI model

Use OpenAI Realtime API. OpenAI's Realtime API supports voice-agent sessions where the app sends audio/text and receives model responses, tool calls, and session events. OpenAI also recommends WebRTC for browser/mobile voice apps because of more consistent performance.  OpenAI Develo... +1

Approach B: Modular pipeline

Use separate components:

Customer voice → STT → LLM → TTS → customer hears answer

For example, Deepgram provides streaming speech-to-text for voice agents, including real-time conversational STT, and ElevenLabs provides text-to-speech streaming for real-time audio output.  Deepgram +2

3. Understand customer intent

The AI should classify why the customer is calling:

- Complaint
- Order status
- Refund
- Claim status
- Technical issue
- Billing issue
- Cancellation request
- New inquiry
- Escalation request

4. Verify customer identity

Before giving private information, the system should verify:

- Mobile number

- OTP
- Policy/order/customer ID
- Date of birth
- Last 4 digits of registered number
- CRM/customer database match

5. Use company knowledge

The AI should answer from your company data:

- FAQ documents
- Policy documents
- Product manuals
- SOPs
- Refund rules
- Claim rules
- Pricing plans
- Internal knowledge base

This is where you use **RAG**: Retrieval-Augmented Generation.

6. Take backend actions

The AI should not only answer. It should perform actions through tools:

- Create ticket
- Check order status
- Check policy status
- Book appointment
- Update complaint
- Send SMS/email
- Schedule callback
- Transfer to human
- Mark priority
- Add call summary

7. Human handoff

The AI must transfer to a human when:

- Customer is angry
- AI confidence is low
- Sensitive/legal/financial issue
- Customer asks for human
- Payment/refund approval needed
- Complaint is high priority

- Verification fails
- Call becomes abusive or complex

8. Dashboard for support team

Admin/support team should see:

- Live calls
 - Call transcript
 - AI summary
 - Sentiment
 - Intent
 - Ticket status
 - Call recording
 - Escalation reason
 - Agent performance
 - Resolution rate
 - Average handling time
 - Failed automation cases
-

3. Production Architecture

 Mermaid



Module 2: Realtime Voice Engine

You have two architecture choices.

Option A: OpenAI Realtime Voice Agent

Best for natural conversation and faster POC.

Flow:

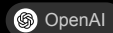
```
Caller audio → Realtime API → AI response audio → Caller
```



Advantages:

- Lower engineering complexity
- More natural speech-to-speech interaction
- Supports tool calls
- Good for demo and production prototype

OpenAI announced Realtime API general availability with features for production voice agents, including SIP phone calling, remote MCP servers, and image inputs.



Option B: Separate STT + LLM + TTS Pipeline

Best when you want more control.

Flow:

```
Caller audio → Deepgram STT → LLM → ElevenLabs TTS → Caller audio
```



Advantages:

- You can swap STT/TTS providers
- Better cost control
- Easier transcript-level debugging
- More flexible for Indian languages/accent tuning

Deepgram documents streaming speech-to-text and voice agent APIs, while ElevenLabs supports streaming text-to-speech.



5. AI Agent Design

Do not build one open-ended AI agent. Build a **controlled multi-layer agent**.

Agent 1: Call Intake Agent

Job:

- Greet customer
- Detect language
- Ask reason for call
- Extract basic details
- Check whether customer is existing or new

Example:

“Hello, this is Akshronix AI support assistant. This call may be recorded for quality and support purposes. How can I help you today?”

Agent 2: Verification Agent

Job:

- Verify customer before sharing private data
- Ask OTP/customer ID/policy number/order ID
- Check backend database
- Block sensitive info if verification fails

Important: Never allow AI to reveal private details before verification.

Agent 3: Intent Detection Agent

Job:

Classify the customer issue.

Example output:

 JSON 

```
{
  "intent": "claim_status",
  "confidence": 0.91,
  "entities": {
    "policy_number": "POL12345",
    "claim_id": "CLM9981"
  },
  "sentiment": "neutral",
  "urgency": "medium"
}
```

Agent 4: Knowledge Agent

Job:

- Search company documents
- Find answer from FAQ/SOP/policy
- Generate grounded answer
- Mention only verified information

Use vector database:

- pgvector
 - Pinecone
 - Weaviate
 - Qdrant
 - ChromaDB for POC
-

Agent 5: Action Agent

Job:

Call backend tools.

Example tools:

JSON



```
[  
  "get_customer_profile",  
  "get_order_status",  
  "get_claim_status",  
  "create_ticket",  
  "schedule_callback",  
  "send_sms",  
  "transfer_to_human",  
  "update_ticket_priority"  
]
```

The AI should not directly access the database. It should call controlled backend functions.

Agent 6: Escalation Agent

Job:

Decide if the call should go to a human.

Escalation rules:

```
IF customer_sentiment = angry
OR confidence < 0.70
OR issue_type = legal/financial/high-risk
OR customer asks for human
OR verification fails 3 times
THEN transfer_to_human()
```



Agent 7: Call Summary Agent

After call ends, generate:

</> JSON



```
{
  "call_summary": "Customer asked about claim CLM9981. AI verified customer and informed
  "intent": "claim_status",
  "sentiment": "neutral",
  "resolution_status": "resolved",
  "ticket_created": false,
  "next_action": "No action required",
  "customer_satisfaction_signal": "positive"
}
```

6. Production Call Flow

1. Customer calls company number
2. AI answers with disclosure
3. AI detects language
4. AI asks reason for call
5. AI extracts intent and entities
6. If private data needed, AI verifies customer
7. AI searches knowledge base or backend system
8. AI gives answer or performs action
9. If issue is complex, AI transfers to human
10. System stores transcript, summary, recording, intent, sentiment, ticket info
11. Dashboard updates for support team



7. Database Design

Use PostgreSQL for production.

Main Tables

customers

</> SQL



```
customers (  
  id UUID PRIMARY KEY,  
  full_name TEXT,  
  phone_number TEXT UNIQUE,  
  email TEXT,  
  customer_type TEXT,  
  language_preference TEXT,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
)
```

calls

</> SQL



```
calls (  
  id UUID PRIMARY KEY,  
  customer_id UUID REFERENCES customers(id),  
  phone_number TEXT,  
  call_direction TEXT, -- inbound/outbound  
  status TEXT, -- completed, failed, transferred  
  started_at TIMESTAMP,  
  ended_at TIMESTAMP,  
  duration_seconds INT,  
  recording_url TEXT,  
  transcript TEXT,  
  summary TEXT,  
  sentiment TEXT,  
  intent TEXT,  
  resolution_status TEXT,  
  escalation_reason TEXT,  
  created_at TIMESTAMP  
)
```

call_messages

</> SQL



```
call_messages (  
  id UUID PRIMARY KEY,  
  call_id UUID REFERENCES calls(id),  
  speaker TEXT, -- customer, ai, human_agent  
  message TEXT,
```

```
timestamp TIMESTAMP,  
confidence FLOAT  
)
```

tickets

SQL



```
tickets (  
  id UUID PRIMARY KEY,  
  customer_id UUID REFERENCES customers(id),  
  call_id UUID REFERENCES calls(id),  
  title TEXT,  
  description TEXT,  
  priority TEXT,  
  status TEXT,  
  assigned_to UUID,  
  created_by TEXT, -- ai, human  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP  
)
```

knowledge_documents

SQL



```
knowledge_documents (  
  id UUID PRIMARY KEY,  
  title TEXT,  
  source_type TEXT,  
  file_url TEXT,  
  content TEXT,  
  version TEXT,  
  is_active BOOLEAN,  
  created_at TIMESTAMP  
)
```

audit_logs

SQL



```
audit_logs (  
  id UUID PRIMARY KEY,  
  actor_type TEXT, -- ai, human, system  
  actor_id TEXT,  
  action TEXT,  
  entity_type TEXT,  
  entity_id UUID,  
  before_data JSONB,
```

```
after_data JSONB,  
created_at TIMESTAMP  
)
```

ai_tool_calls

SQL



```
ai_tool_calls (  
  id UUID PRIMARY KEY,  
  call_id UUID REFERENCES calls(id),  
  tool_name TEXT,  
  input JSONB,  
  output JSONB,  
  status TEXT,  
  error_message TEXT,  
  created_at TIMESTAMP  
)
```

8. Recommended Tech Stack

POC Stack

Good for fast demo:

```
Frontend: Next.js or React  
Backend: FastAPI / Node.js  
Voice: Twilio Media Streams  
AI: OpenAI Realtime API or OpenAI + Deepgram + ElevenLabs  
Database: PostgreSQL  
Vector DB: pgvector / ChromaDB  
Cache/Queue: Redis  
Storage: S3-compatible storage  
Dashboard: Next.js Admin Panel  
Deployment: Docker + AWS/GCP/Azure
```



Production Stack

```
Frontend:  
- Next.js  
- Tailwind CSS  
- Role-based dashboard  
  
Backend:  
- Python FastAPI or Node.js NestJS  
- WebSocket server for realtime audio
```



- REST APIs for dashboard
- Background workers using Celery/BullMQ

AI Layer:

- OpenAI Realtime API for voice
- RAG pipeline for knowledge base
- Tool-calling layer for CRM/ticket/order systems
- Guardrail engine

Database:

- PostgreSQL
- Redis
- pgvector/Qdrant/Pinecone
- Object storage for recordings

Infra:

- Docker
- Kubernetes
- Nginx/API Gateway
- Prometheus + Grafana
- Sentry
- OpenTelemetry
- CI/CD with GitHub Actions

9. API Design

Call APIs

 http 

```
POST /api/calls/inbound
GET /api/calls/:id
GET /api/calls/:id/transcript
GET /api/calls/:id/summary
POST /api/calls/:id/escalate
```

Ticket APIs

 http 

```
POST /api/tickets
GET /api/tickets
GET /api/tickets/:id
PATCH /api/tickets/:id/status
PATCH /api/tickets/:id/assign
```

Knowledge APIs

⌘ http



```
POST /api/knowledge/upload
GET /api/knowledge
POST /api/knowledge/search
PATCH /api/knowledge/:id
```

AI Tool APIs

⌘ http



```
POST /api/tools/get-customer
POST /api/tools/get-order-status
POST /api/tools/get-claim-status
POST /api/tools/create-ticket
POST /api/tools/send-sms
POST /api/tools/schedule-callback
POST /api/tools/transfer-call
```

10. Safety and Guardrails

This is very important for production.

The AI must not:

- Promise refund unless backend confirms
- Share personal data without verification
- Give legal/medical/financial advice as final authority
- Abuse or argue with customer
- Invent company policies
- Bypass escalation rules
- Store unnecessary sensitive data
- Continue call if customer opts out of recording where opt-out is required

Guardrail examples

Rule 1:

If user asks for account-specific data, verify identity first.

Rule 2:

If confidence is below 0.70, say:

"I want to make sure I give you the correct information. Let me connect you to a su



Rule 3:

If user asks for refund approval:

AI can create request but cannot approve refund unless backend policy allows.

Rule 4:

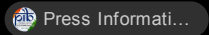
If customer becomes angry:

Apologize once, summarize issue, transfer to human.

11. Compliance Notes for India

Because you are in India and may build for Indian businesses/government, include compliance from the start.

The DPDP Act, 2023 deals with processing digital personal data. The official Act requires consent requests to be presented in clear and plain language, with access in English or languages listed in the Eighth Schedule, and with contact details of the Data Protection Officer or authorized person where applicable. 

For commercial/promotional calls, also be careful about TRAI rules on unsolicited commercial communication. TRAI and PIB announced amendments in February 2025 to strengthen complaint mechanisms against spam calls/messages. 

Production checklist:

- Announce AI identity where required: “You are speaking with an AI assistant.”
- Announce call recording where applicable.
- Store consent event.
- Store purpose of recording.
- Encrypt recordings and transcripts.
- Define retention period.
- Allow deletion/export where applicable.
- Avoid using call data for AI training unless explicit permission exists.
- Keep audit logs for all AI actions.
- Separate service calls from promotional/marketing calls.
- Consult legal expert before real deployment.

12. Admin Dashboard Screens

Your software should have these screens:

1. Live Calls Dashboard

Shows:

- Active calls
- Customer phone
- Current intent
- Sentiment
- AI confidence
- Call duration
- Escalation button

2. Call History

Shows:

- Call recording
- Transcript
- Summary
- Sentiment
- Resolution status
- Created ticket
- Agent notes

3. Ticket Dashboard

Shows:

- AI-created tickets
- Priority
- SLA risk
- Assigned agent
- Status
- Customer details

4. Knowledge Base Manager

Shows:

- Uploaded documents
- FAQ entries
- SOP versions
- Document status
- Last updated date

5. AI Performance Analytics

Shows:

- Total calls

- Resolved by AI
- Transferred to human
- Failed calls
- Average response latency
- Most common intents
- Customer sentiment trends
- Top unanswered questions

6. Call Quality Review

Shows:

- Random call samples
 - AI hallucination flag
 - Wrong answer flag
 - Escalation missed flag
 - Human review comments
-

13. MVP Scope

Do not build everything at once. Start with a controlled POC.

MVP Version

Build this first:

1. Customer calls demo number
2. AI greets customer
3. AI understands intent
4. AI answers from FAQ
5. AI creates ticket if needed
6. AI transfers to human for complex issue
7. Dashboard shows transcript and summary

MVP Use Case Example

Choose one domain first:

Best POC Domain: Insurance Customer Support

Supported intents:

- Policy status
- Claim status
- Premium due date
- Document requirement

- Complaint registration
- Human callback request

Why insurance is good:

- Clear workflows
 - Many repetitive calls
 - Need human escalation
 - Good enterprise value
 - Easy to demo with dummy data
-

14. Production Roadmap

Phase 1: POC

Duration target: small team can build with limited scope.

Features:

- Twilio call integration
- AI voice conversation
- FAQ-based answers
- Basic ticket creation
- Transcript and summary
- Simple dashboard

Phase 2: MVP

Features:

- Customer verification
- CRM integration
- RAG knowledge base
- Sentiment detection
- Priority detection
- Human handoff
- Call recording
- Audit logs

Phase 3: Production V1

Features:

- Multi-tenant SaaS
- Role-based access

- SLA management
- Agent workload dashboard
- Advanced analytics
- Call quality review
- Consent management
- Monitoring and alerting

Phase 4: Enterprise Version

Features:

- SIP trunk support
- On-prem/private cloud deployment
- Custom AI models
- Multilingual support
- WhatsApp + email + chat support
- Advanced compliance
- Human-in-loop approval
- Integration marketplace

15. Important Production Metrics

Track these from day one:

```
AI Resolution Rate
Human Transfer Rate
Average Handling Time
First Call Resolution
Customer Sentiment
AI Confidence
Hallucination Rate
Tool Call Failure Rate
Latency per AI response
Call Drop Rate
Escalation Accuracy
Cost per Call
```



Target for first production version:

```
AI resolution rate: 40-60%
Human escalation: 30-50%
Wrong answer rate: <3%
Average AI response delay: <1.5-2.5 sec
Tool failure rate: <1%
```



16. Folder Structure

```
ai-voice-support-agent/
|
├─ apps/
|   ├── web-dashboard/
|   └─ voice-gateway/
|
├─ backend/
|   ├── app/
|   |   ├── api/
|   |   ├── calls/
|   |   ├── tickets/
|   |   ├── customers/
|   |   ├── knowledge/
|   |   ├── ai/
|   |   |   ├── agents/
|   |   |   ├── prompts/
|   |   |   ├── tools/
|   |   |   ├── guardrails/
|   |   |   └─ rag/
|   |   ├── telephony/
|   |   ├── audit/
|   |   └─ config/
|   └─ tests/
|   └─ migrations/
|
├─ infra/
|   ├── docker/
|   ├── kubernetes/
|   └─ terraform/
|
├─ docs/
|   ├── PRD.md
|   ├── TRD.md
|   ├── ARCHITECTURE.md
|   ├── API_SPEC.md
|   └─ SECURITY.md
|
└─ README.md
```



17. Example AI System Prompt

Use this type of prompt for the voice agent:

You are an AI customer support voice agent for the company.



Your goals:

1. Understand the customer's issue.
2. Verify identity before sharing private information.
3. Answer only from approved company knowledge or backend tools.
4. Create a ticket when the issue cannot be solved.
5. Escalate to a human when confidence is low, customer is angry, or the issue is s

Rules:

- Always be polite, short, and clear.
- Do not reveal private customer data without verification.
- Do not invent policy, pricing, refund, or claim information.
- If you are unsure, ask one clarifying question.
- If still unsure, escalate to human.
- For every important action, call the correct tool.
- At the end of the call, generate a structured call summary.

Tone:

Professional, calm, helpful, and human-like.

Never say:

- "I am just an AI."
- "I cannot help you."

Instead say:

- "Let me check that for you."
- "I will connect you to a support specialist."

18. Best Version of This Product

The strongest product angle is not “AI that talks”.

The best product is:

AI Voice Support Operations Platform for businesses, where AI handles repetitive customer calls, creates tickets, updates CRM, summarizes calls, and escalates complex cases to humans.

This makes it useful for real companies because it saves support cost and improves response time.

19. Final Recommended Build Strategy

Start with this version:

```
Domain: Insurance customer support
Call Provider: Twilio
AI Voice: OpenAI Realtime API
Backend: FastAPI or Node.js
Database: PostgreSQL
Knowledge: pgvector / ChromaDB
Dashboard: Next.js
Deployment: Docker on AWS/GCP
```



Build only 6 intents first:

1. Policy status
2. Claim status
3. Premium due date
4. Required documents
5. Complaint creation
6. Human callback request



This will be enough to show a strong POC to a company or department. Then expand into full SaaS.